

へんすう 変数

JavaScriptの基礎：データの入れ物「変数」をマスターしよう

JavaScriptを学ぶ上で、最初の重要なステップが「変数」の理解です。

留学生の皆さんが日本語を学ぶとき、単語を覚えて整理するように、プログラミングでもデータを整理して名前をつける作業が必要です。変数を使いこなせるようになると、プログラムの中でデータを自由に再利用（さいりよう）し、複雑な処理を組み立てることができるようになります。

では、まず、「変数とは何か」から、はじめましょう。

I. 変数とは何か：データを保存する「名前付きの箱」

プログラミングにおける「変数」とは、プログラムの中で扱う数値や文字列などのデータを保存しておくための「名前のついた箱」のようなものです。

空港で荷物を預けるシーンを想像してみてください。たくさんのスーツケースがあるとき、『どれが誰のものか？』『中身が何か？』を見分けるために「名前のラベル」を貼りますよね。これが変数です。ラベル（変数名）があるからこそ、後で必要なときに迷わずそのデータを取り出すことができます。

1. 「代入」

「名前のついた箱」である「変数」の中にデータを入れることを、プログラミングでは「代入」と言います。

- 名前（ラベル）：変数名（例: `score`, `myName`）
- 箱の中身：データ（例: 数字の `100`, 文字の `"山田"`）

代入するときは、`=`（イコール）という記号を使います。ここで一番大切なルールがあります。

「右側（みぎがわ）にあるデータを、左側（ひだりがわ）の変数に入れる」,

数学（すうがく）の「`=`」は「左と右が同じ」という意味ですが、プログラミングの「`=`」は「入れる」という動きを表します。

たとえば、テストの点数を`score`という変数に保存したいときは次のように書きます。

```
score = 100;
```

- これは、「100」という数字を、「score」という名前の箱に入れた」という意味になります。

また、「再代入（さいだいにゅう）」とは、「一度、変数に値を代入したあとで、別の値を入れなおす」ことです。

一度、100という点数を入れたscoreを、あとで80点に変更する場合、下のように入ります。

```
score = 100; // 最初に100を代入（だいにゅう）します
// プログラムの途中は省略します
score = 80; // 一度、100を入れた変数（へんすう）scoreに、80を再代入（さいだいにゅう）します
```

2. 変数宣言とキーワードの種類

変数を利用するには、まずシステムにその変数名を使うことを知らせる「変数宣言」が必要です。現代のJavaScriptでは、主に **const** と **let** という2つのキーワードを使用して宣言を行います。

なぜ変数を宣言するキーワードが2つあるのでしょうか？ 実は、JavaScriptには2種類の箱があります。プログラムを効率的に、そしてミスなく作るためには、これらを使い分けることが重要です。

キーワードは、プログラムが動いている途中で「値を変更するかどうか」で使い分けます。

letで宣言した変数は、あとで値を変えることができます。つまり「再代入が可能」です。反対に、**const**で宣言した変数は、あとで値を変えることができません。

種類	特徴	使いどき	例
let	中身を入れ替えられる箱	ゲームのスコアやクリックされた回数など、後で値が変わるもの	const MAX_LEVEL = 99;
const	一度入れたら変えられない箱	名前、誕生日、設定値など、ずっと同じもの	let score = 0;（あとで score = 10; と変えられる）

💡 プロの視点：なぜ **const** を優先するのか？

const で宣言された変数は、後から値を変更できません。一度決めた値が「変わらない」ことが約束されいてると、後からコードを読み返したときに迷わず、バグ（ミス）を減らすことができるからです。

ですから、みなさんは、できるだけ **const** を使いましょう。そして、後から値を変更しなければならないときだけ **let** を使いましょう。

※ むかしは **let**の代わりに **var** が使われていましたが、知らないうちに変数の中身が書き換えられてしまうかもしれないので、現在は、「使わないのがよい」とされています。

3. 変数の初期化

変数を宣言する際、同時に値を代入することを「初期化（しよきか）」と呼びます。

- 宣言と初期化を同時に行う例:

```
let myDog = "Rover"; // 宣言（せんげん）と同時に "Rover" という文字列を入れる
const pi = 3.14;      // constは宣言（せんげん）時に必ず初期化（しよきか）が必要
```

- ^{せんげん}宣言だけ先に行う例（**let**のみ可能）：

```
let myName; // 箱だけ用意する。この時の中身は undefined（未定義（みていぎ））となる
myName = "Chris"; // 後から値を代入（だいにゅう）する
```

4. ^{へんすう}変数の名前の付け方

^{へんすう}変数に名前を付けるときには、いくつかの守るべきルールと慣習（かんしゅう）があります。

- **大文字と小文字は区別される**: **Name** と **name** は別の^{へんすう}変数として扱われます。
- **使用できる文字**: ^{はんかくえいすうじ}半角英数字、アンダースコア（`_`）、ドル記号（`$`）が使えますが、**数字から始めることはできません**。
- **キャメルケース (camelCase)**: 複数の単語（つな）を繋げる場合、2つ目以降の単語の先頭を大文字にする^か書き方が一般的です。
 - ^{れい}例: `finalOutputValue`、`userName`
- **意味のある名前**: `a` や `b` といった一文字ではなく、`count` や `isRaining` など、**中身が何であるか伝わる英語の名前**を付けるのがよい、とされています。

⚠ 留学生への重要アドバイス：^{はんかく}半角と^{ぜんかく}全角

JavaScriptのコードを書く際、最も多いエラーの原因は^{ぜんかく}全角文字の使用です。^{へんすう}変数の名前にも、^{ぜんかく}全角文字は使えません。すべて^{はんかく}半角文字にしてください。

箱の使い方がわかったところで、次は、その箱に入れる「データの種類」について詳しく見ていきましょう。

以下のようなものがあります。

- **文字列型**: `let name = "山田";`（シングル `' '` または ^{いんようふ}ダブル `" "` の引用符で囲む）
- **数値型**: `let age = 17;`（引用符は付けない）
- **論理型**: `let iAmAlive = true;`（`true` または `false` の^{しんぎ}真偽値）
- **配列**: `let myNameArray = ["Chris", "Bob", "Jim"];`（複数の値をまとめて管理するリスト）
- **オブジェクト**: `let dog = { name: "ポチ", breed: "ダルメシアン" };`（関連するデータと機能をひとつにまとめたもの）

ここでは「文字列型」「数値型」「論理型」について説明します。「配列」と「オブジェクト」については、あとの章で説明します。

II. 文字列型 (String)：テキストデータを扱う

ウェブページで「こんにちは」というメッセージを表示したり、ユーザーの名前を扱ったりする際に使われるのが「文字列（もじれつ）」です。

1. 文字列を作る

文字列を作るには、テキストを引用符（クォーテーション）で囲みます。

- シングルクォーテーション：'こんにちは'
- ダブルクォーテーション："こんにちは"

どちらを使っても構いませんが、ひとつのプログラムの中で統一するのが良い習慣です。

2. テンプレートリテラル：最新のスマートな書き方

バッククォート（`）と \${} を使った「テンプレートリテラル」を使うと、変数の埋め込みや改行が非常に簡単になります。

```
const name = "佐藤";
// 変数(へんすう)を文章の中に埋(う)め込(こ)む
const greeting = `こんにちは、${name}さん!`;
console.log(greeting);
```

実際、どんなふうになるか見てみましょう。F12 キー（ノートPCは Fn + F12）を使ってコンソールを開いてから、下の「実行」ボタンを押してみてください。

<button onclick="const name = '佐藤'; const greeting = ``javascript const name = "佐藤"; // 変数(へんすう)を文章の中に埋(う)め込(こ)む const greeting = こんにちは、\${name}さん!; console.log(greeting);

実際、どんなふうになるか見てみましょう。F12 キー（ノートPCは Fn + F12）を使ってコンソールを開いてから、下の **「実行」** ボタンを押してみてください。

```
<button onclick="const name = '佐藤'; const greeting = `こんにちは、${name}さん!`;
console.log(greeting);">実行</button>
```

```
````javascript
// テンプレートリテラルなら、そのまま改行(かいぎょう)もできます
const multiLine = `一行目です
二行目です
三行目です`;
console.log(multiLine);
```

下の「実行」ボタンを押すと、上のコードの実行結果がコンソールに表示されます。

<button onclick="const multiLine = ``javascript // テンプレートリテラルなら、そのまま改行（かいぎょう）もできます const multiLine = 一行目です 二行目です 三行目です; console.log(multiLine);

下の \*\*「実行」\*\* ボタンを押すと、上のコードの実行結果がコンソールに<ruby>表示<rt>ひょうじ</rt></ruby>されます。

```
<button onclick="const multiLine = `一行目です
二行目です
三行目です`;
console.log(multiLine);">実行</button>
```

<!--a href="./html/03\_01\_templit.html">ここをクリックして<ruby>例<rt>れい</rt></ruby>をみてみましょう。</a-->

文字の次は、計算に欠かせない「<ruby>数値<rt>すうち</rt></ruby>」の扱いに進みます。

\*\*\*

### ## III. <ruby>数値<rt>すうち</rt></ruby>型 (Number) と特別な値 (NaN / Infinity)

JavaScriptでは、整数（100）も小数（3.14）も、すべて同じ「<ruby>数値<rt>すうち</rt></ruby>」型として扱います。

#### ### 1. <ruby>算術<rt>さんじゅつ</rt></ruby><ruby>演算<rt>えんざん</rt></ruby>子（計算の記号）

<ruby>数値<rt>すうち</rt></ruby>を使って、算数と同じように計算ができます。

\* `+` (<ruby>足<rt>た</rt></ruby>し<ruby>算<rt>ざん</rt></ruby>) `−` (<ruby>引<rt>ひ</rt></ruby>き<ruby>算<rt>ざん</rt></ruby>) `\*` (<ruby>掛<rt>か</rt></ruby>け<ruby>算<rt>ざん</rt></ruby>) `/` (<ruby>割<rt>わ</rt></ruby>り<ruby>算<rt>ざん</rt></ruby>) `%` (余り)

※ <ruby>算術<rt>さんじゅつ</rt></ruby><ruby>演算<rt>えんざん</rt></ruby>子については、あとで、もっと詳（くわ）しく説明します。

#### ### 2. 特別な<ruby>数値<rt>すうち</rt></ruby>：NaNとInfinity

<ruby>数値<rt>すうち</rt></ruby>型には、注意が必要な特殊な値があります。

\* \*\*NaN (Not a Number)\*\* : 「<ruby>数値<rt>すうち</rt></ruby>ではない」状態です。<ruby>例<rt>たと</rt></ruby>えば、文字列を無理やり計算しようとした時に発生します。  
\* \*\*Infinity\*\* : 無限に大きな<ruby>数値<rt>すうち</rt></ruby>です。

計算の次は、プログラムが「正しいか間違いか」を判断するためのデータを見てみましょう。

\*\*\*

## ## IV. `<ruby>論理<rt>ろんり</rt></ruby>`型 (Boolean) : YesかNoかの判断基準

プログラムが「もし～なら、これをする」という`<ruby>制御<rt>せいぎょ</rt></ruby>` (せいぎょ) を行う際、その`<ruby>土台<rt>どだい</rt></ruby>`となるのが「`<ruby>論理<rt>ろんり</rt></ruby>` (ろんり)」型です。

### ### 1. 2つの値だけ

`<ruby>論理<rt>ろんり</rt></ruby>`型には、true (`<ruby>真<rt>しん</rt></ruby>`／正しい) と false (`<ruby>偽<rt>ぎ</rt></ruby>`／間違い) の2つの値しか存在しません。

### ### 2. 比較と`<ruby>組<rt>く</rt></ruby>`み`<ruby>合<rt>あ</rt></ruby>`わせ

比較`<ruby>演算<rt>えんざん</rt></ruby>`子を使うと、結果として`<ruby>論理<rt>ろんり</rt></ruby>`型の値が得られます。

```
* score === 100 (スコアは100と<ruby>厳密<rt>げんみつ</rt></ruby>に等しいか?)
* age >= 20 (年齢は20以上か?)
```

また、`<ruby>論理<rt>ろんり</rt></ruby>``<ruby>演算<rt>えんざん</rt></ruby>`子を使って複数の条件を`<ruby>組<rt>く</rt></ruby>`み`<ruby>合<rt>あ</rt></ruby>`わせることもできます。

```
|<ruby>演算<rt>えんざん</rt></ruby>子|意味|覚えるポイント| |
|---|---|---|---|
|&&|～かつ～ (AND)|すべて true のときだけ true|
||\||～または～ (OR)|どちらかが true であれば true|
|!|～ではない (NOT)|true と false を逆転させる|
```

※ 比較`<ruby>演算<rt>えんざん</rt></ruby>`子と`<ruby>論理<rt>ろんり</rt></ruby>``<ruby>演算<rt>えんざん</rt></ruby>`子については、次の章で、もっと詳(くわ)しく説明します

`<ruby>論理<rt>ろんり</rt></ruby>`型を正しく理解すると、複雑なシステムの判断ロジックを整理して、ユーザーの`<ruby>入力<rt>にゅうりょく</rt></ruby>`に対して「正しく`<ruby>反応<rt>はんのう</rt></ruby>`はんのう`<ruby>する</ruby>`」プログラムを書くことができるようになります。

\*\*\*

## ## V. `<ruby>未<ruby>定義<rt>ていぎ</rt></ruby>``<rt>みていぎ</rt></ruby>`型 (Undefined) : 値が決まっていない状態

`<ruby>変数<rt>へんすう</rt></ruby>`を作ったけれど、中身をまだ何も入れていないとき、JavaScriptはその状態を undefined と表現します。これは「`<ruby>未<ruby>定義<rt>ていぎ</rt></ruby>``<rt>みていぎ</rt></ruby>` (みていぎ)」型という特殊な型になります。「`<ruby>未<ruby>定義<rt>ていぎ</rt></ruby>``<rt>みていぎ</rt></ruby>`」とは「まだ`<ruby>定義<rt>ていぎ</rt></ruby>`されていない」「まだ、どういうものか決まっていない、わからない」という意味です。

\*\*「空の箱」と「存在しない箱」の違い\*\*

`<ruby>初心者<rt>しょしんしゃ</rt></ruby>`の方が間違いやすいのが、以下の違いです。

- \* undefined：箱（`<ruby>変数<rt>へんすう</rt></ruby>`）はあるけれど、中身がまだ入っていない状態。
- \* ReferenceError：そもそも箱（`<ruby>変数<rt>へんすう</rt></ruby>`）`<ruby>自体<rt>じたい</rt></ruby>`が存在していない、または名前を間違えている状態。

💡 デバッグのヒント 実行結果に undefined と`<ruby>表示<rt>ひょうじ</rt></ruby>`されたら、「データの`<ruby>代入<rt>だいにゅう</rt></ruby>`を忘れていないか？」「スペルミスをしていないか？」を疑ってみましょう。これはミスを特定するための重要な手がかりになります。

\*\*\*

## ## VI. 型の`<ruby>変換<rt>へんかん</rt></ruby>` (Type Conversion)

Webサイトの`<ruby>入力<rt>にゅうりよく</rt></ruby>`フォームから受け取るデータは、数字に見えても実際には「文字列」です。これを使って計算するには「型の`<ruby>変換<rt>へんかん</rt></ruby>`」が必要です。

### ### 1. データの抽出：parseInt()

文字列の中から数字だけを取り出す便利な機能があります。

```
console.log(parseInt("55kg")); // 55 （数字の部分だけ取り出す） console.log(parseInt("12.99ドル")); // 12
（整数部分だけ取り出す） console.log(parseInt("A123")); // NaN （先頭が文字だと失敗する）
```

### ### 2. 安全な`<ruby>数値<rt>すうち</rt></ruby>``<ruby>入力<rt>にゅうりよく</rt></ruby>`の受け取りフロー

プロの現場では、以下のように段階を踏んでデータを処理します。

1. 受け取る：prompt() など文字列として`<ruby>入力<rt>にゅうりよく</rt></ruby>`を受け取る。
2. `<ruby>変換<rt>へんかん</rt></ruby>`する：Number() や parseInt() で`<ruby>数値<rt>すうち</rt></ruby>`に変える。
3. チェックする：isNaN() で、正しく数字になったか確認する。

### ### 3. 🧐 学習者の味方：typeof `<ruby>演算<rt>えんざん</rt></ruby>`子

「今のこの`<ruby>変数<rt>へんすう</rt></ruby>`は、何型なんだろう？」と迷ったときは、typeof を使いましょう。

```
const value = "123"; console.log(typeof value); // "string" と表示（ひょうじ）される
```

```
const numValue = Number(value); console.log(typeof numValue); // "number" と表示（ひょうじ）される
```



自分の書いたコードが思ったとおりに動いているか、コンソールで `typeof` を使って確認するクセをつけましょう。

\*\*\*

## ## VII. まとめ：プログラミングの基礎を固めよう

変数へんすうと型の知識(ちしき)は、これからの学習の土台どだい(どだい)になります。

### 💡 \*\*最重要ポイント\*\*

- \* 変数へんすうは「名前のついた箱」：データを記憶(きおく)きおく(きおく)し、何度でも使うために必要です。
- \* 基本は `const` を使う：安全のために `const` を使い、値を変えるときだけ `let` を使しましょう。
- \* 数値すうち(すうち)変換へんかん(へんかん) (すうちへんかん) を忘れずに： `prompt()` などを受け取ったデータは `Number()` や `parseInt()` で数字に変えてから計算しましょう。

### ⚠️ \*\*初心者(しよしんしゃ)が間違えやすいポイント\*\*

- \* 引用符(いんようふ) (クォーテーション) の忘れ：文字を書くときは必ず " " で囲んでください。囲まないと変数へんすう(へんすう)だと勘違(かんちが)いされてエラーになります。
- \* 全角(ぜんかく) (ぜんかく)文字の使用：プログラムのコード (`let`, `const`, `;` など) は、必ず半角(はんかく) (はんかく)の英数字(えいすうじ) (えいすうじ)で書いてください。全角(ぜんかく) (ぜんかく)のスペースがひとつあるだけでも、プログラムは動かなくなります。

これで変数へんすう(へんすう)の基本(きほん)は完璧(かんぺき) (かんぺき) (かんぺき)です。さあ、次に進みましょう！